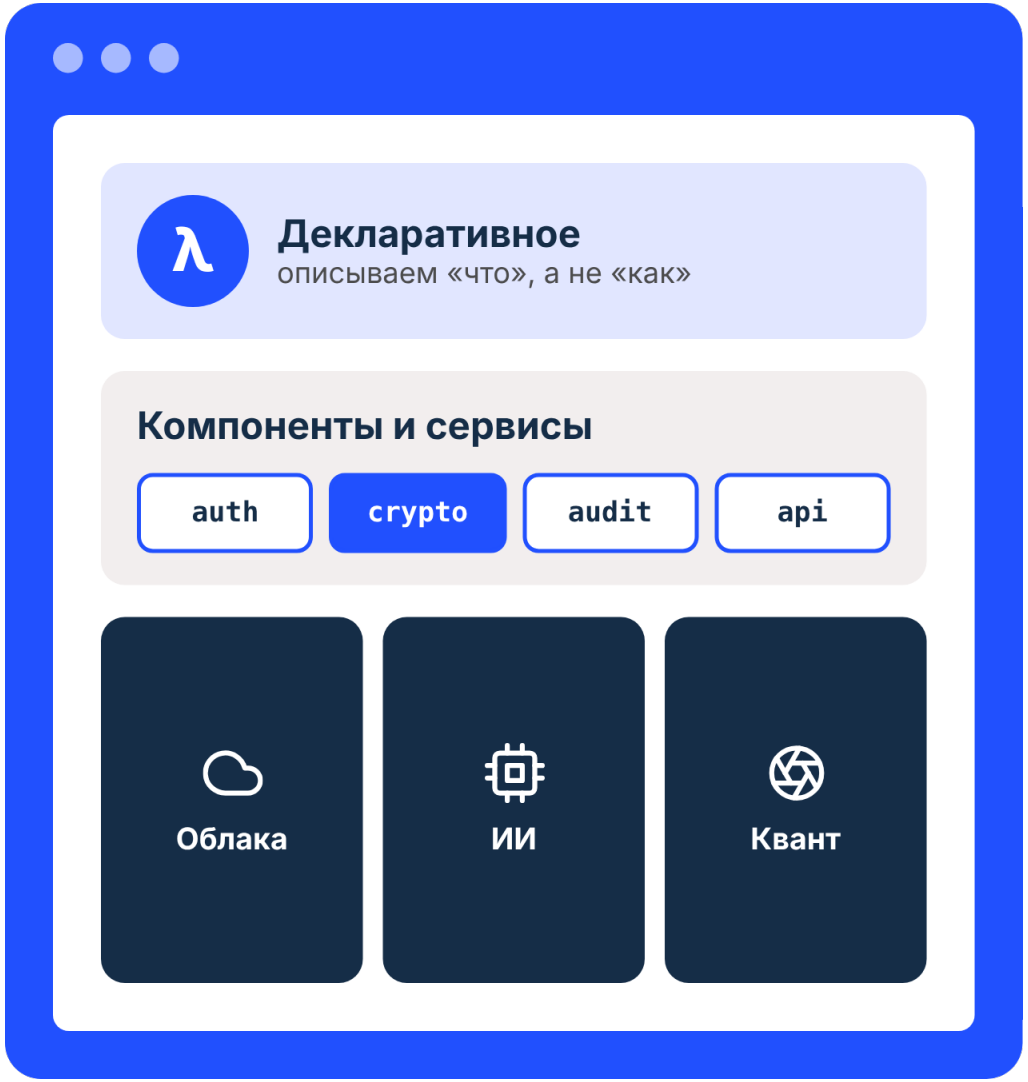
Лекция 2

Декларативное программирование, компонентные технологии и перспективы

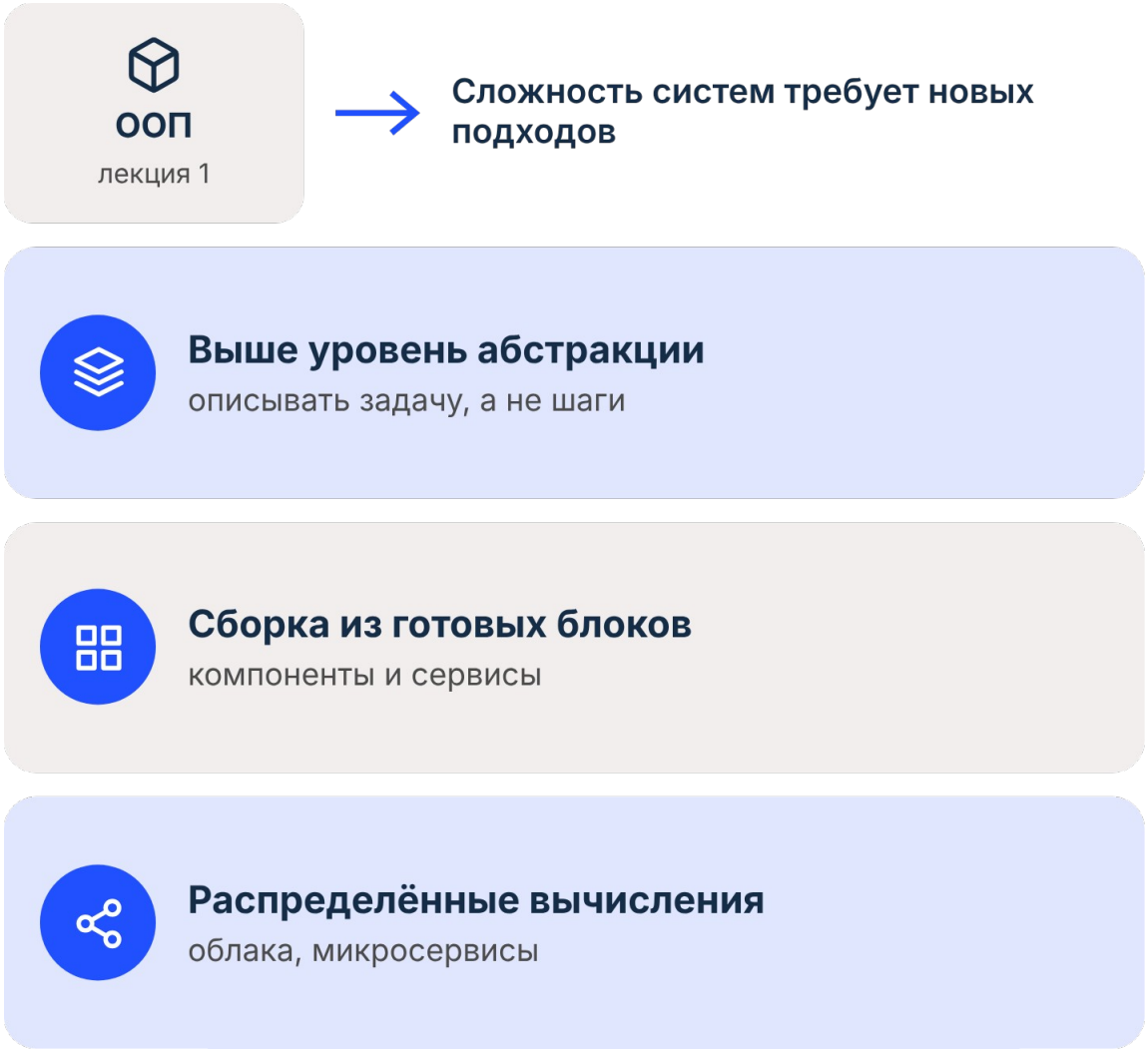
Функциональное и логическое программирование, компоненты и микросервисы, облака, ИИ и квантовые вычисления



Эволюция не остановилась на ООП

В лекции 1 мы прошли путь от «спагетти-кода» до объектно-ориентированных систем.

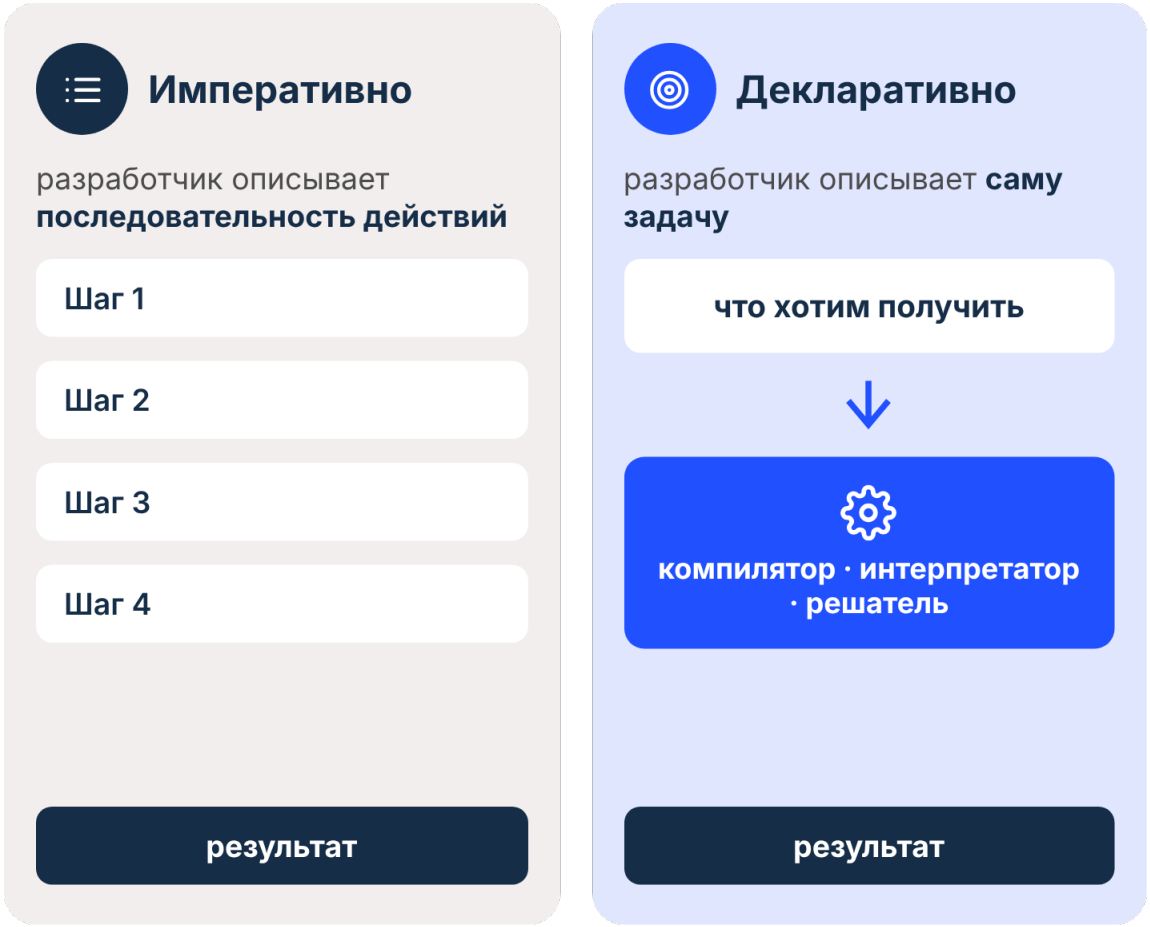
Сложность современных систем, особенно в ИБ, требует **новых подходов**



Императивный и декларативный подход

Декларативный подход описывает **саму задачу** — что мы хотим получить.

Поиск решения возлагается на исполнительную систему

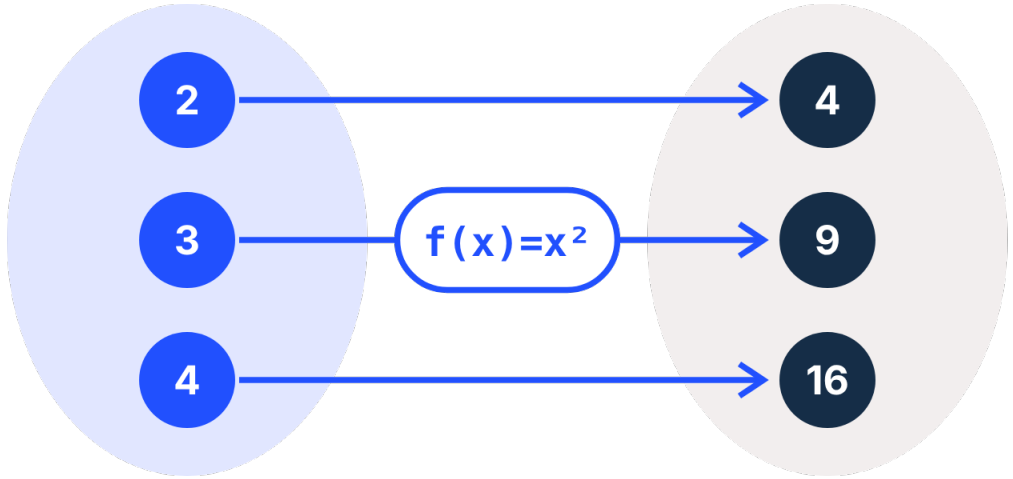


Функциональное программирование

Программа — вычисление значений функций.

Опирается на **математическое понятие функции**

Избегает изменения состояния и побочных эффектов



Результат зависит только от входа

Нет изменения состояния

Нет побочных эффектов

Пять принципов функционального программирования

```
add(2, 3)
↓

5 — всегда
```

Чистые функции

результат — только из аргументов

```
a = (1, 2)
b = a + (3,)
a не изменился
```

Неизменяемость

вместо изменения — новая структура

```
f = len
apply(f, data)
```

Функции — объекты первого класса

передаём, возвращаем, храним в переменных

```
map(f, data)
filter(g, data)
```

Функции высшего порядка

принимают или возвращают функции

```
fact(n) =
n * fact(n-1)
```

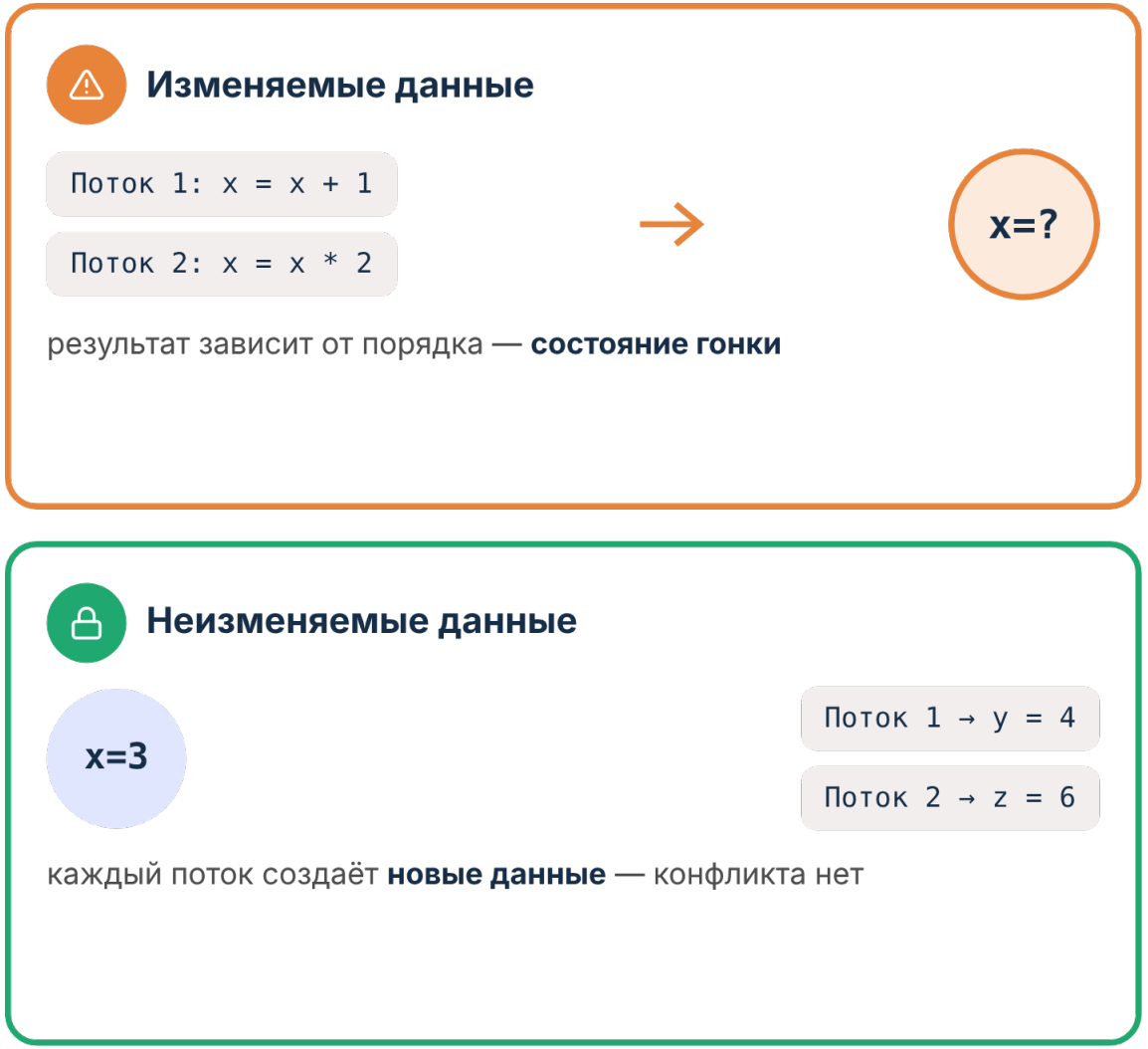
Рекурсия

вместо циклов со счётчиком

Неизменяемость против гонки данных

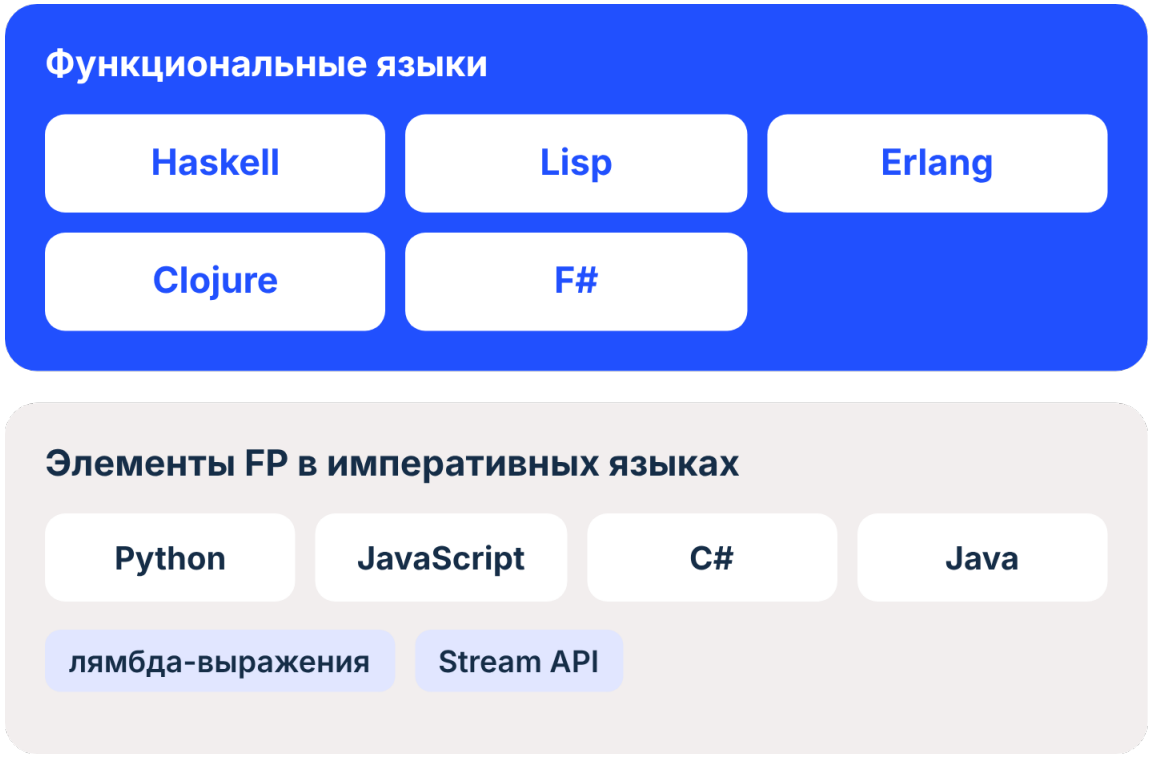
Неизменяемые данные устраняют целый класс ошибок в многопоточных средах.

Данные нельзя изменить конкурентно — **состояние гонки исключено**



Языки функционального программирования

Элементы FP активно проникают в императивные языки.

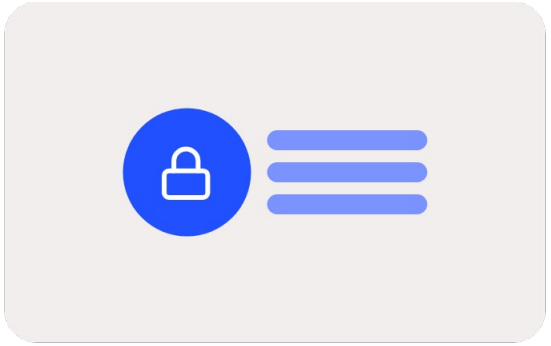


Функциональное программирование и ИБ



Предсказуемость

проще аудит и доказательство корректности



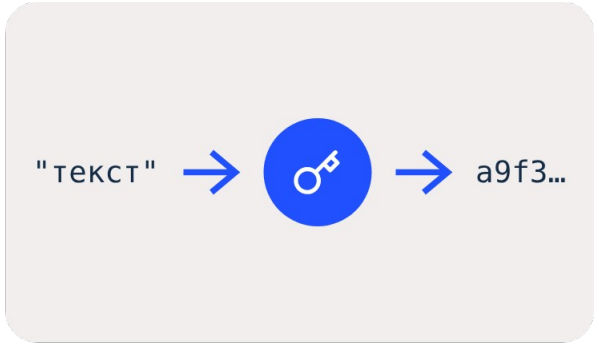
Многопоточность

неизменяемые данные исключают гонку



Парсинг и анализ

сетевые протоколы, логи, форматы данных



Криптография

чистые функции для преобразований

Пример: три стиля на Python

Задача: возвести в квадрат все чётные числа из списка.

Императивный

```
result = []
for x in data:
  if x % 2 == 0:
    result.append(x * x)
```

Функциональный

```
result = list(
 map(lambda x: x * x,
  filter(lambda x:
   x % 2 == 0, data)))
```

Генератор списка

ближе к декларативному

```
result = [
  x * x
  for x in data
  if x % 2 == 0
]
```

Логическое программирование

Задача описывается фактами и правилами, решение система выводит сама.

Основной язык — **Prolog**

Запрос — это цель, которую система доказывает

Факты

```
отец(иван, петр).
родитель(петр, анна).
```

Правило

```
дед(X, Y) :-
 отец(X, Z),
 родитель(Z, Y).
```




```
?- дед(иван, анна).
true. % система сама нашла Z = петр
```

Логическое программирование в ИБ

Политики безопасности можно описать логическими правилами.

Система автоматически проверяет, **не нарушает ли запрос правила**

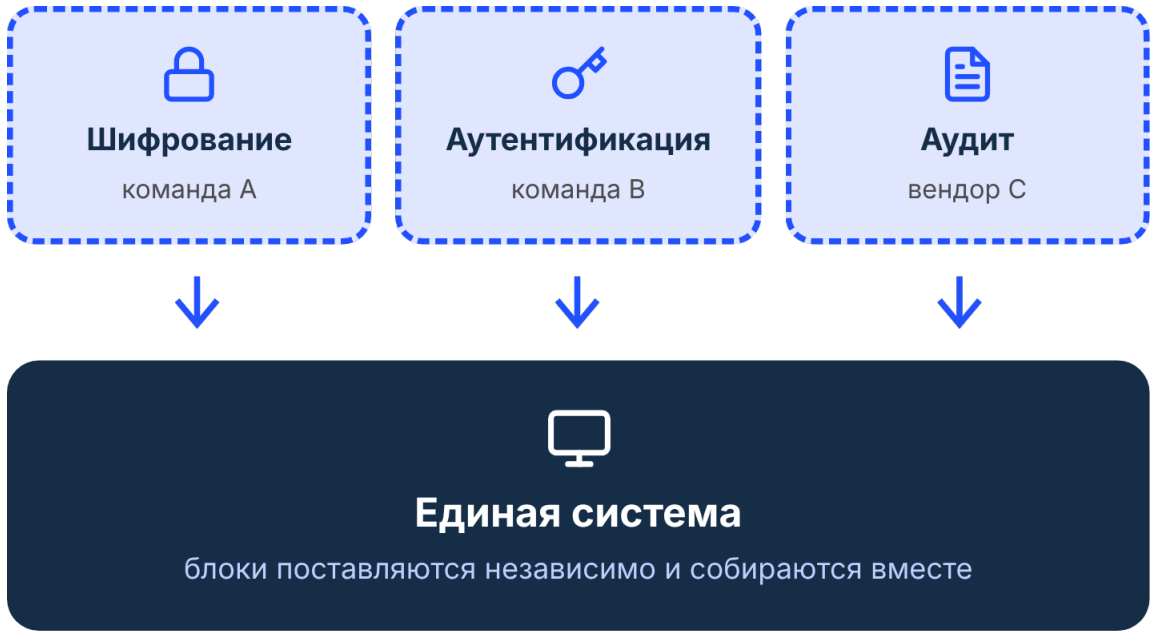


Компонентные технологии

Повторно используется не только код, но и целые функциональные блоки.

Блоки распространяются независимо

Из них собирается единая система

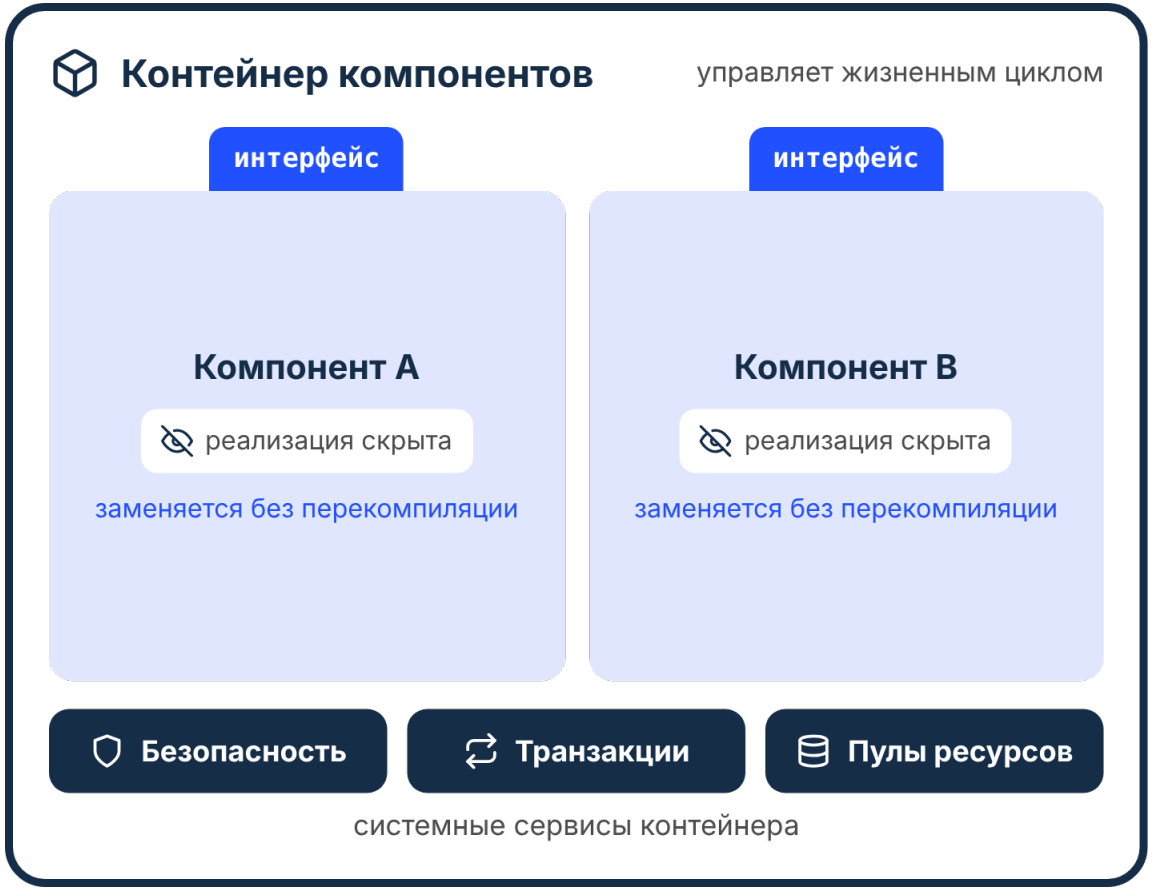


Компонент, контейнер, интерфейс

Компонент — независимо поставляемая единица с интерфейсом и скрытой реализацией

Контейнер — среда выполнения: жизненный цикл и системные сервисы

Интерфейс — контракт: какие методы и свойства доступны



Исторические компонентные технологии

Microsoft

COM / DCOM

- внутри процесса, между процессами и по сети
- широко в Windows

консорциум OMG

CORBA

- объекты на разных языках
- интерфейсы на IDL
- сложна в настройке

платформа Java

JavaBeans

- компонентная модель Java

платформа .NET

.NET Components

- наследуют System.ComponentModel.Component
- размещаются в дизайнере

Сервисно-ориентированная архитектура (SOA)

Приложение — набор слабосвязанных сервисов, взаимодействующих по сети.

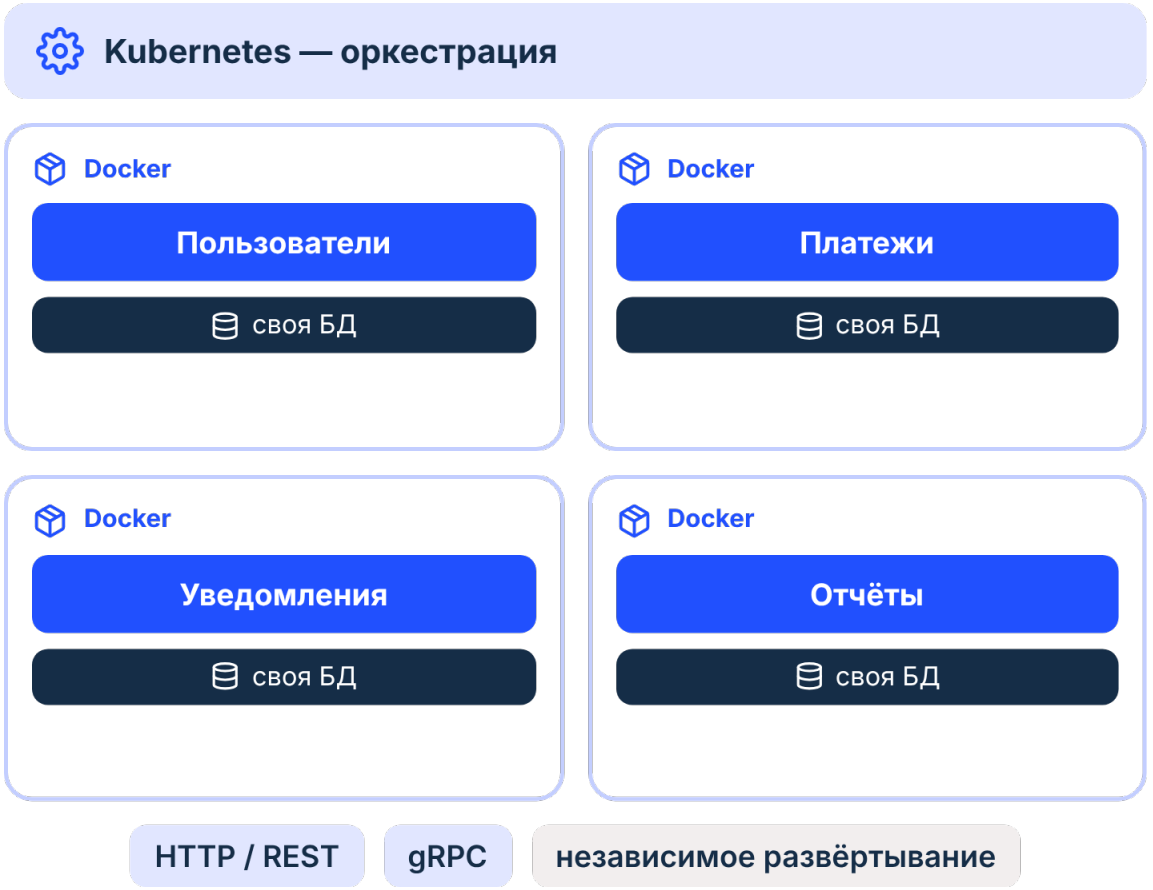
Каждый сервис — **одна бизнес-операция** и чёткий контракт



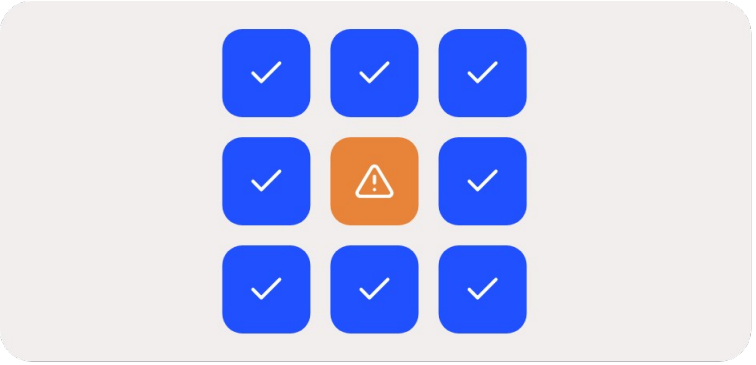
Микросервисная архитектура

Эволюция SOA: сервисы максимально маленькие, автономные и независимо развёртываемые.

Одна функция — один сервис со **своей базой данных**

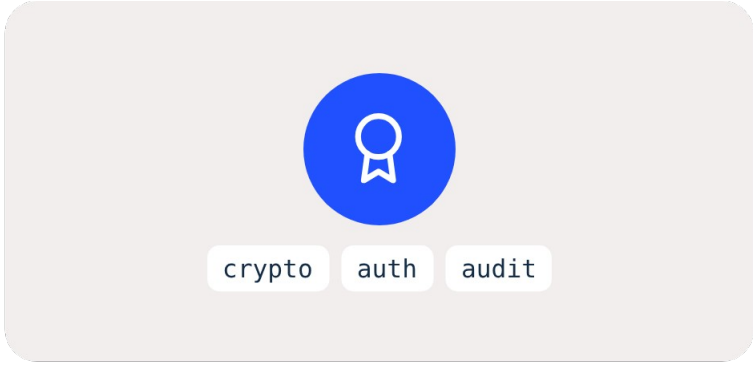


Компоненты и микросервисы: польза для ИБ



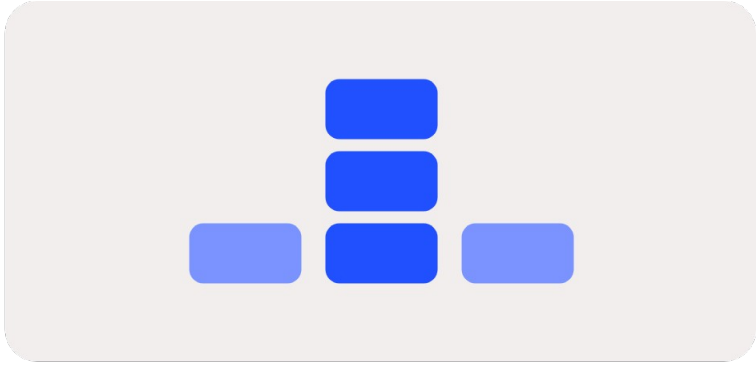
Изоляция

компрометация одного сервиса не захватывает остальные



Проверенные решения

сертифицированные компоненты шифрования, аутентификации, аудита



Масштабируемость и отказоустойчивость

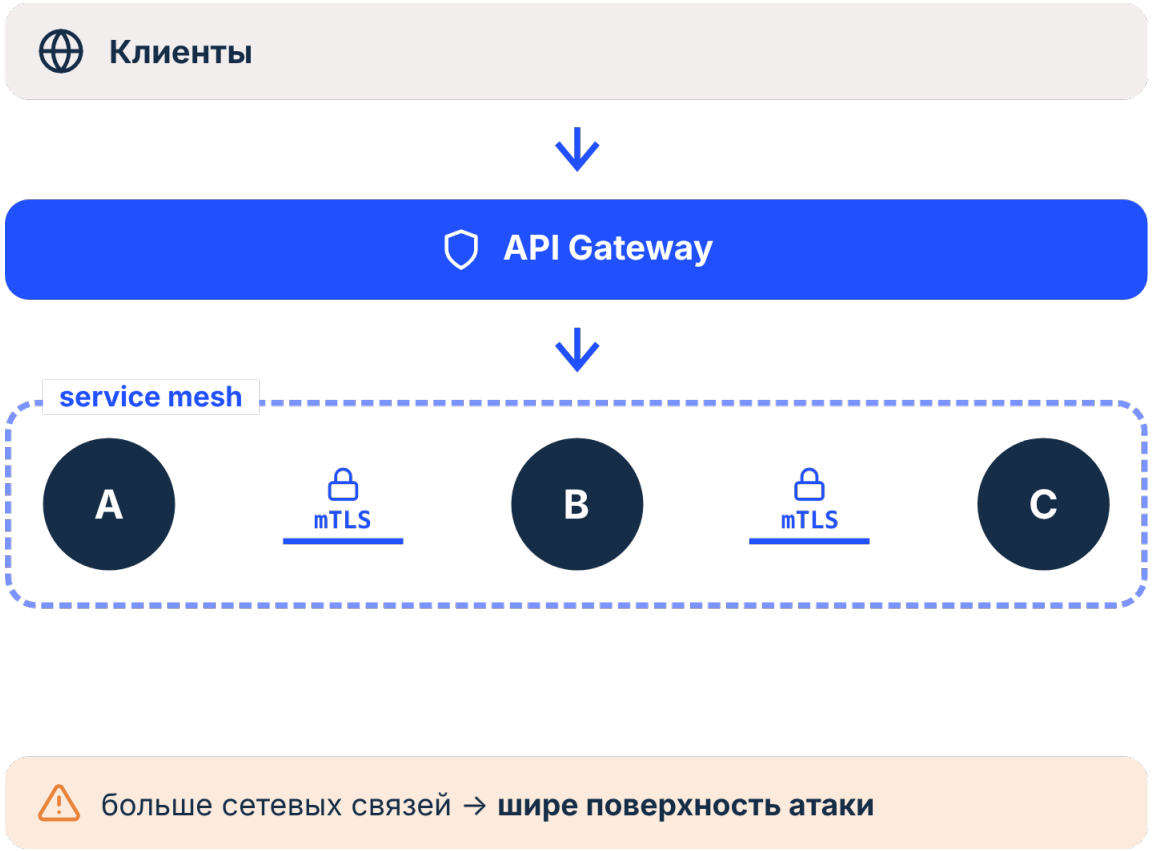
сервисы масштабируются независимо, отказ одного — не отказ системы

Проблемы безопасности микросервисов

Множество сетевых взаимодействий усложняет управление доступом и мониторинг.

Нужны специальные подходы: **API Gateway, service mesh, mTLS**

Строгая политика безопасности



Перспективы развития технологий

Технологии программирования отвечают на вызовы современного мира.



Рост объёмов данных



Высокая скорость разработки



Киберугрозы

Облачные вычисления

Облачные платформы дают готовые сервисы IaaS и PaaS.

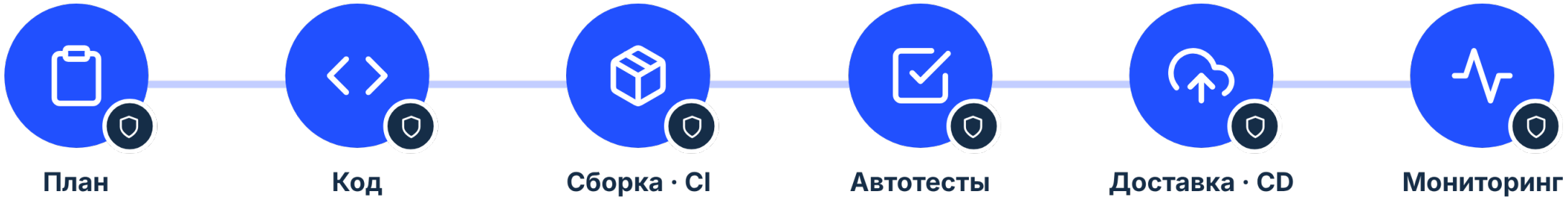
Разработчик сосредоточен на коде, а не на серверах

Для ИБ: **Cloud Security** и модель разделяемой ответственности



DevOps и DevSecOps

DevOps объединяет разработку и эксплуатацию. **DevSecOps** — безопасность на всех этапах.



CI · CD · Инфраструктура как код (IaC) · Автоматизированное тестирование · безопасность на каждом этапе

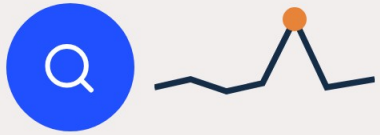
Искусственный интеллект в программировании

```
def check(pwd):
  # предложено ИИ
```

⚠ может быть уязвимым

Генерация кода

GitHub Copilot, ChatGPT — ускоряют, но код нужно проверять



Поиск уязвимостей

ML в статическом и динамическом анализе, аномалии трафика, обнаружение вторжений

```
- query = "…" + user
+ query = "…?", (user,)
```

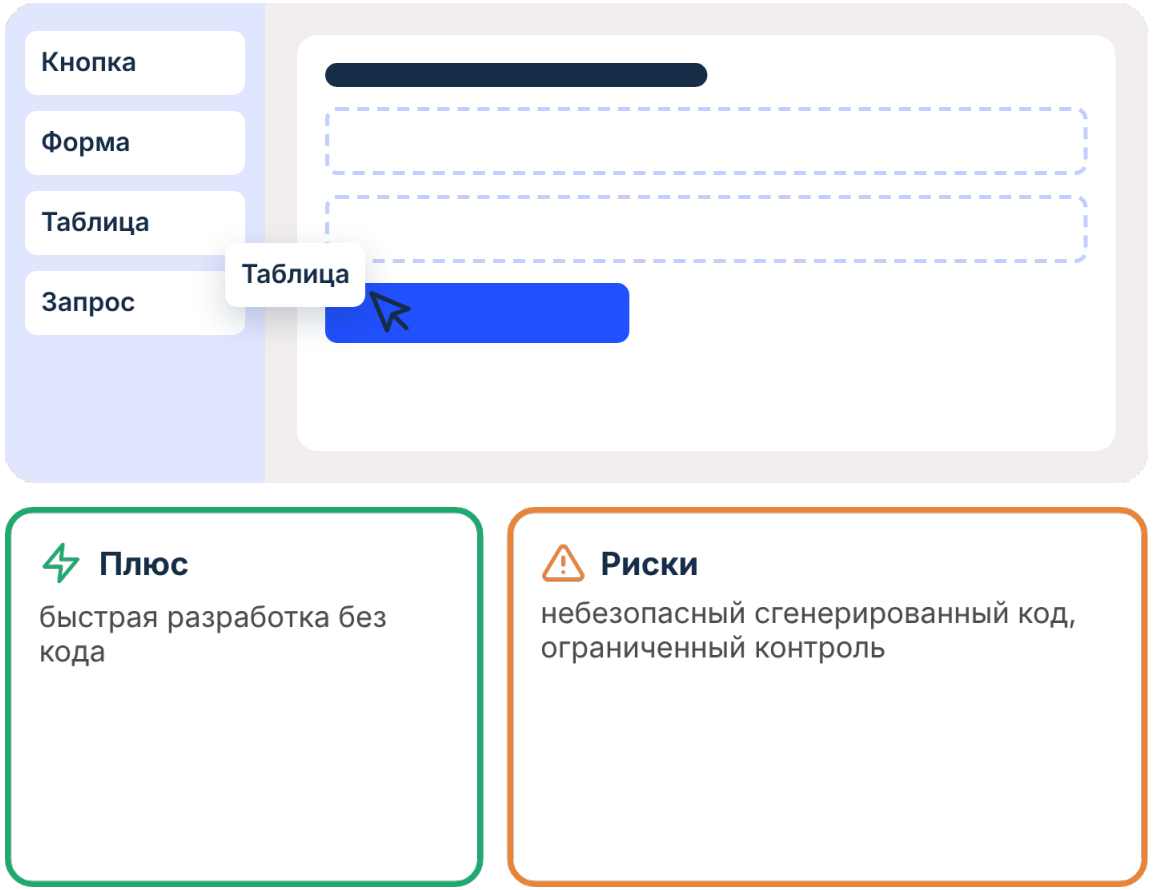
Автоисправление

системы предлагают патчи для найденных уязвимостей

Low-code и No-code

Приложения собираются в визуальных конструкторах — с минимумом кода или без него.

Специалистам по ИБ придётся **адаптировать методы аудита**



Квантовое программирование

С квантовыми компьютерами появляются новые парадигмы программирования.

Программистам будущего понадобится понимание квантовых вычислений



Новые парадигмы

кубиты, запутанность, квантовые алгоритмы

Угроза

алгоритм Шора взламывает существующие криптосистемы

Возможность

квантовая криптография и постквантовые алгоритмы

Безопасность цепочки поставок ПО

Приложения собираются из множества сторонних библиотек и компонентов.

Атаки на цепочку поставок становятся **всё опаснее**



Атаки на цепочку

- компрометация репозитория
- вредоносный код в популярной библиотеке

Защита

- SBOM — анализ состава ПО
- Проверка подписей
- Изолированные среды сборки

Заключение

Эволюция программирования — непрерывное повышение уровня абстракции и автоматизации.

Новые технологии ставят перед специалистами ИБ **сложные задачи**

Выбор правильной парадигмы — залог успешного проекта

